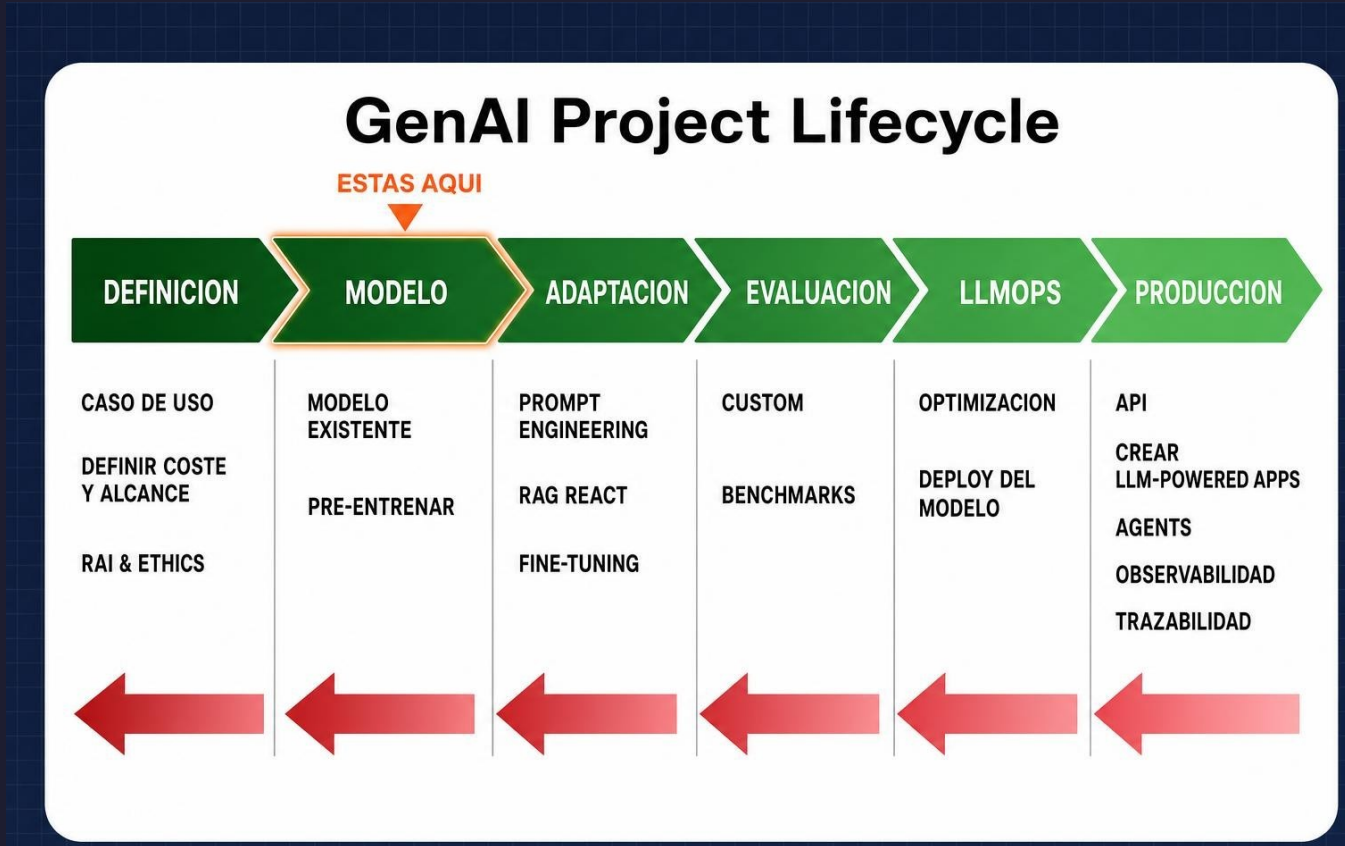
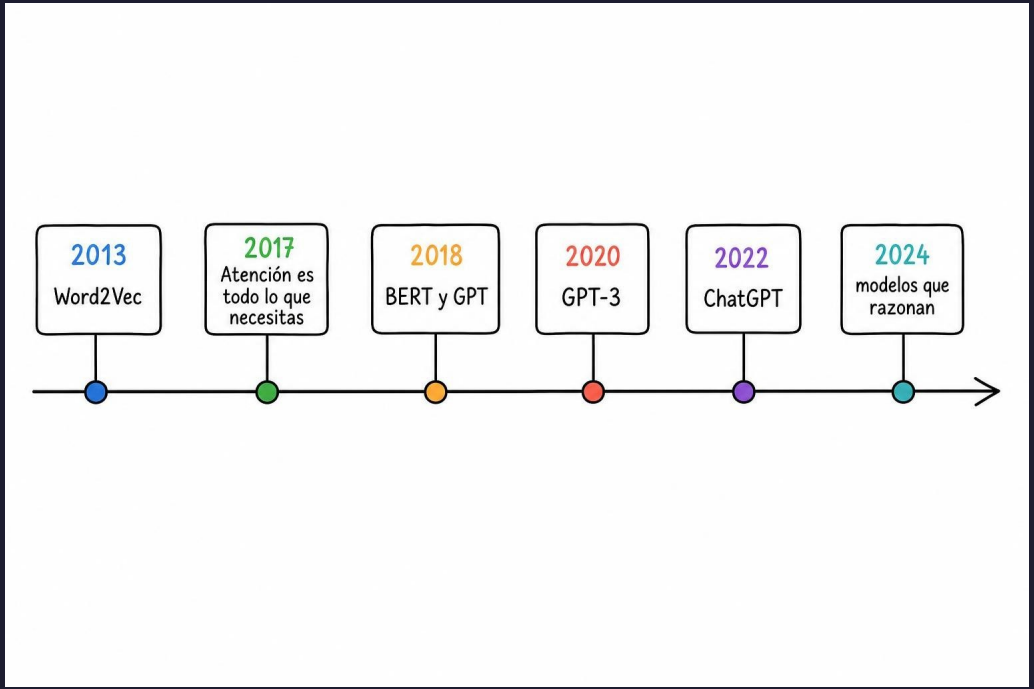
keep coding

AI ENGINEERING

Transformers

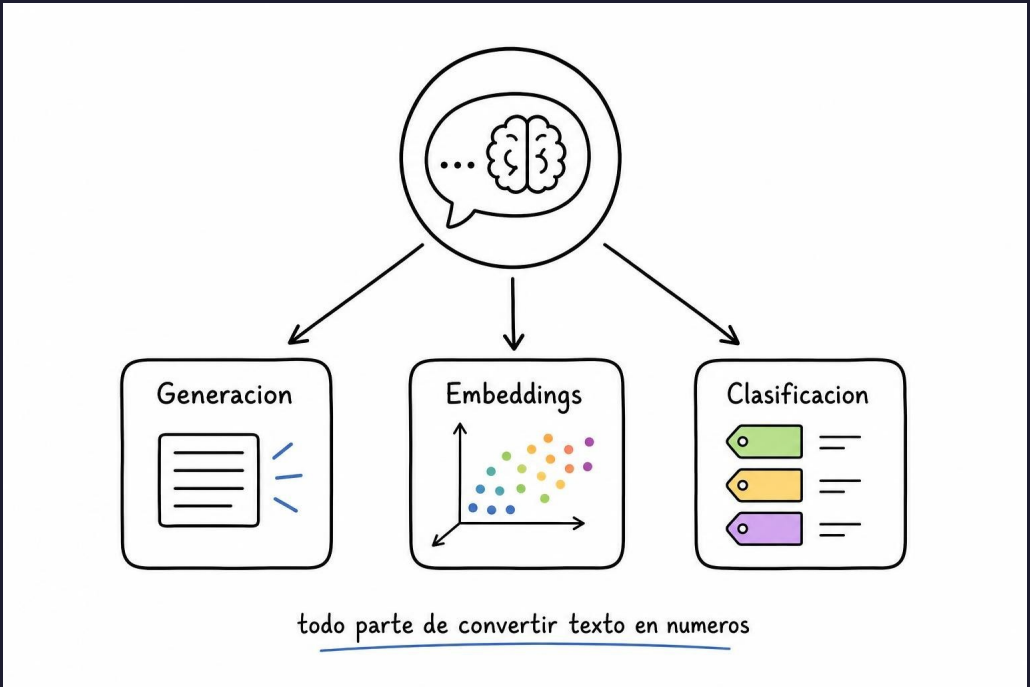

GenAI Project Lifecycle
ESTAS AQUI
DEFINICION
MODELO
ADAPTACION
EVALUACION
LLMOPS
PRODUCCION
CASO DE USO
DEFINIR COSTE Y ALCANCE
RAI & ETHICS
MODELO EXISTENTE
PRE-ENTRENAR
PROMPT ENGINEERING
RAG REACT
FINE-TUNING
CUSTOM
BENCHMARKS
OPTIMIZACION
DEPLOY DEL MODELO
API
CREAR LLM-POWERED APPS
AGENTS
OBSERVABILIDAD
TRAZABILIDAD

De Bag of Words a los LLM



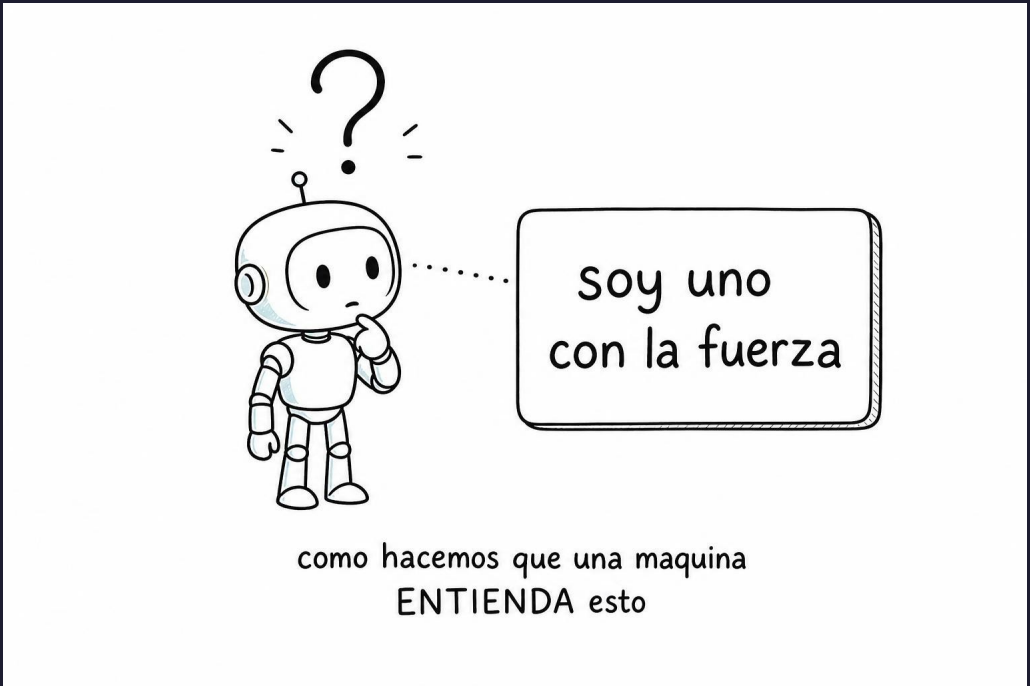
En 10 años pasamos de contar palabras a modelos que razonan.

Language AI



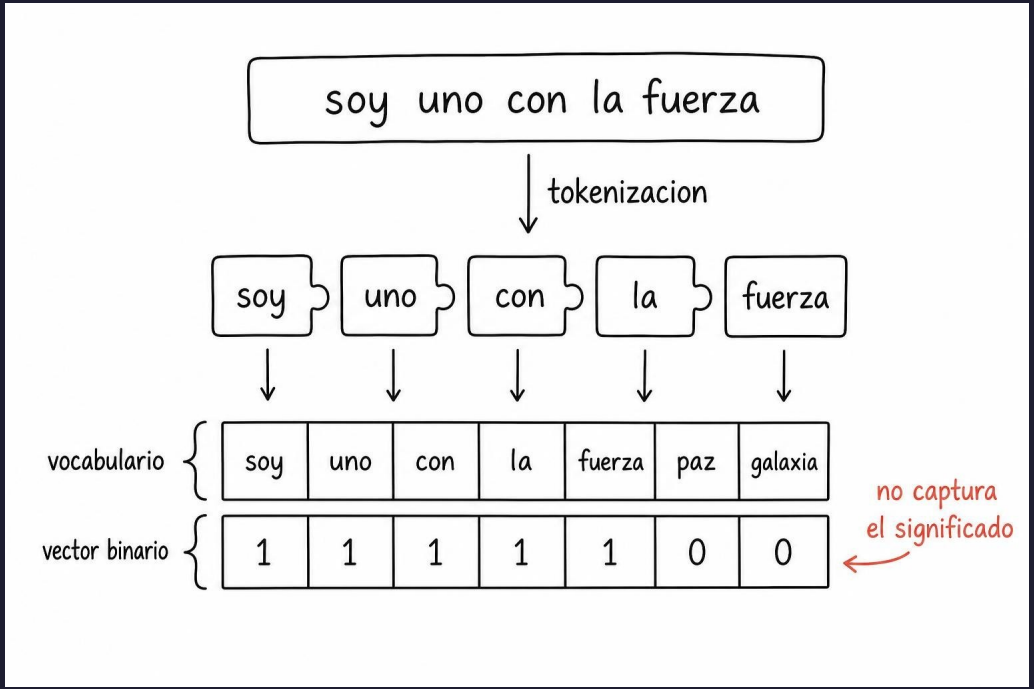
Generar · representar (embeddings) · clasificar.

El reto: entender el lenguaje



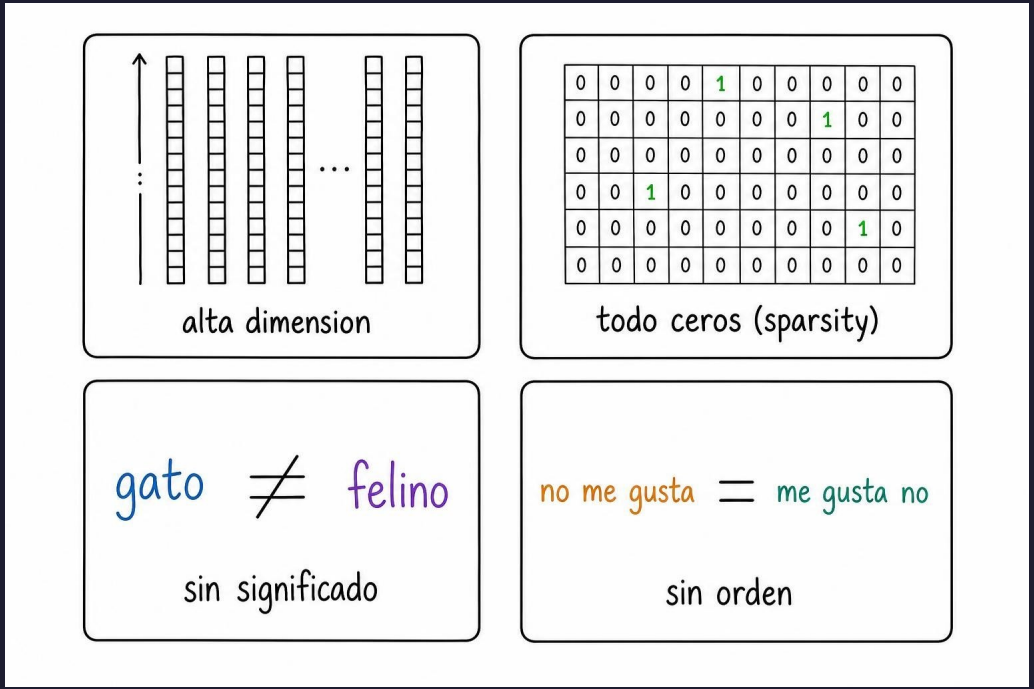
Nuestro hilo de todo el módulo: “soy uno con la fuerza”.

Bag of Words



Texto → números. Pero pierde el significado.

Por qué Bag of Words no basta



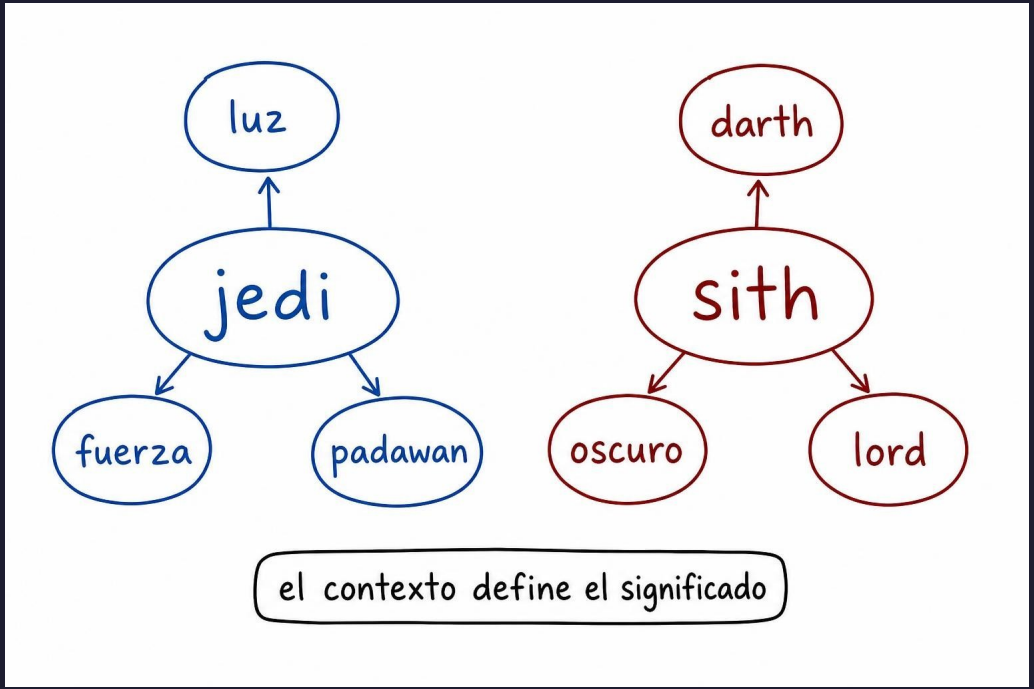
Cuatro problemas que lo hacen inservible para entender.

La hipótesis distribucional

“Si dos palabras aparecen en contextos similares, tienden a tener significados similares.”

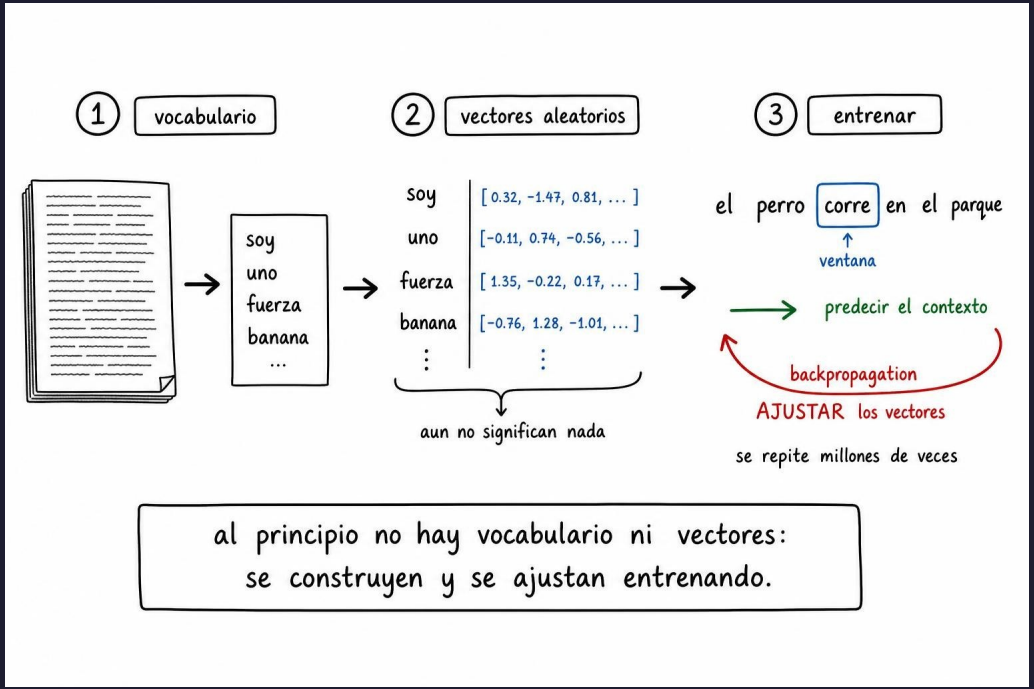
Zellig Harris · “Distributional Structure” · 1954

El contexto define el significado



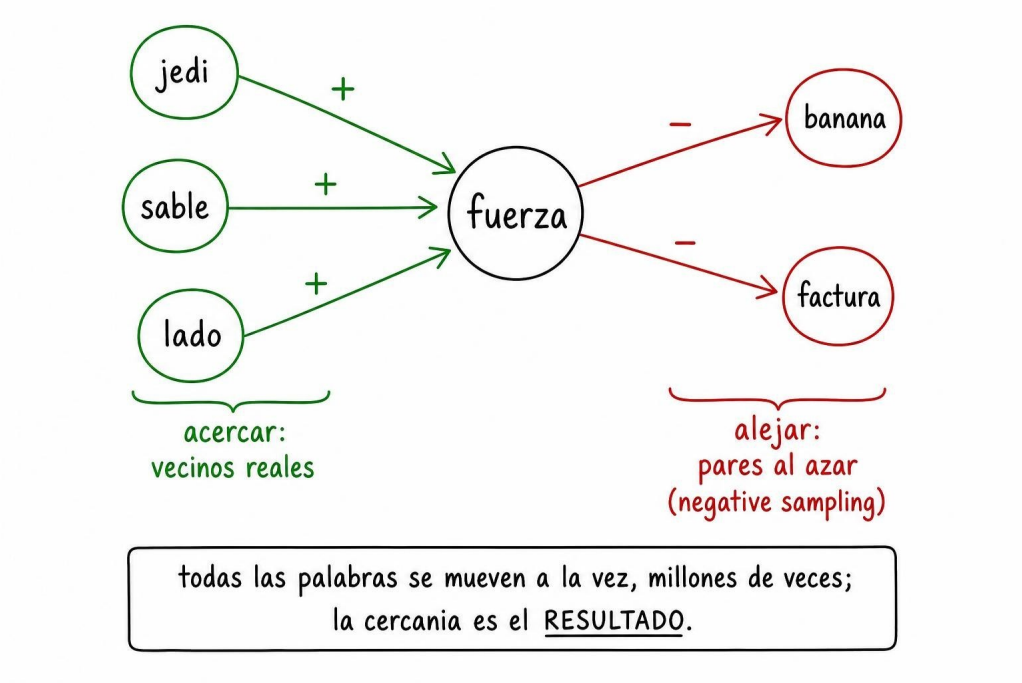
La misma palabra cambia según con quién aparece.

Word2Vec · empieza de cero



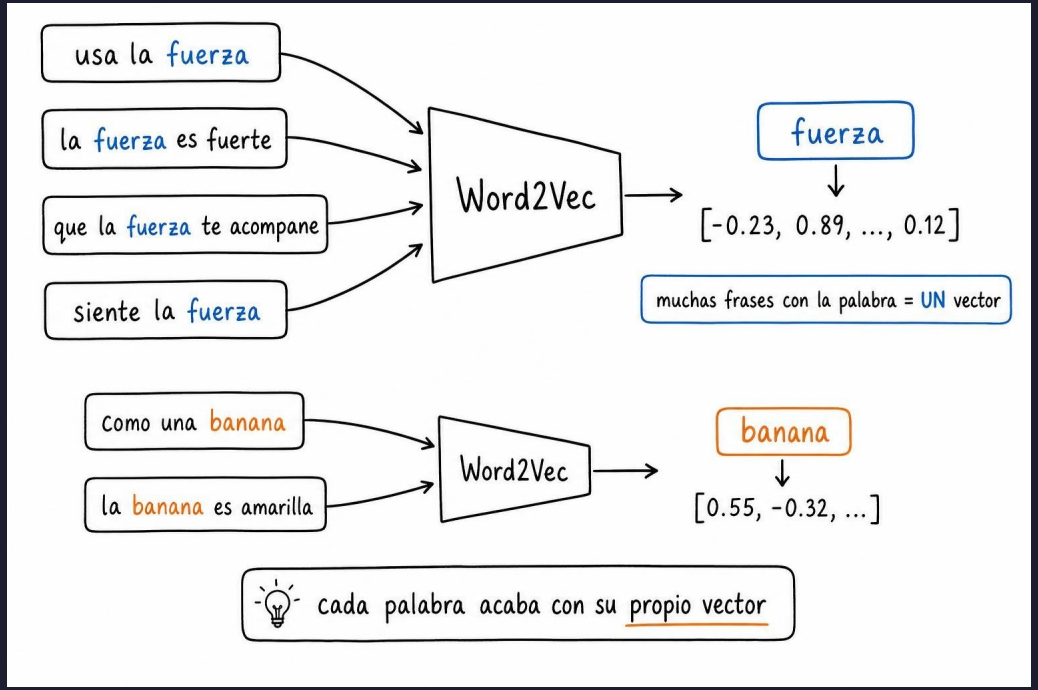
Al principio no hay vocabulario ni vectores: arrancan aleatorios.

Word2Vec · el tira y afloja



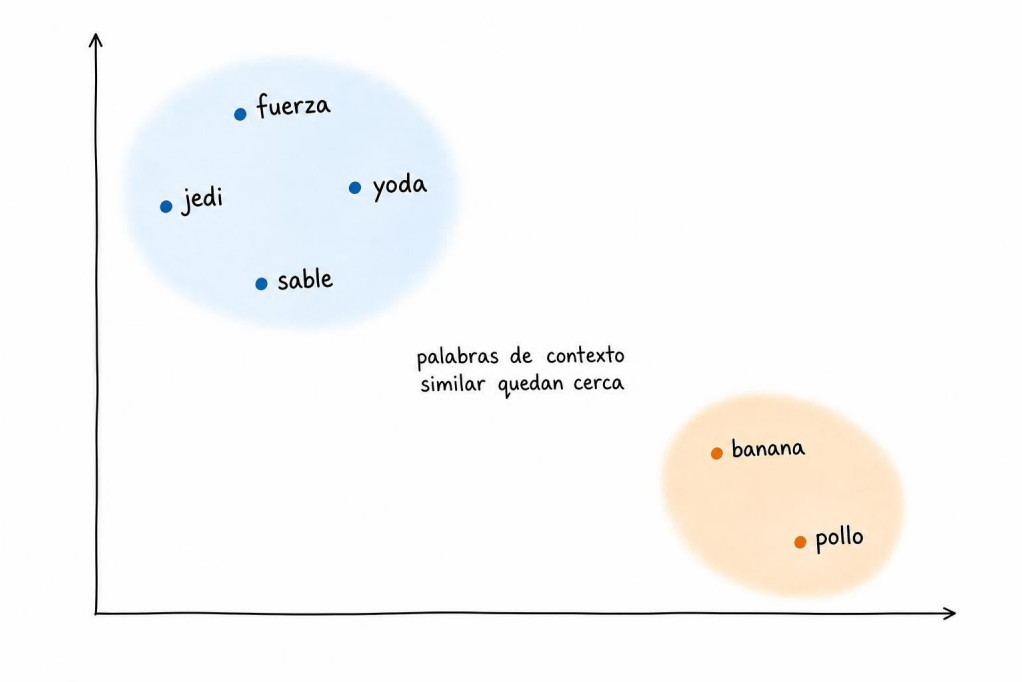
Acerca a los vecinos reales, aleja a los del azar. La cercanía es el RESULTADO.

Word2Vec · un vector por palabra



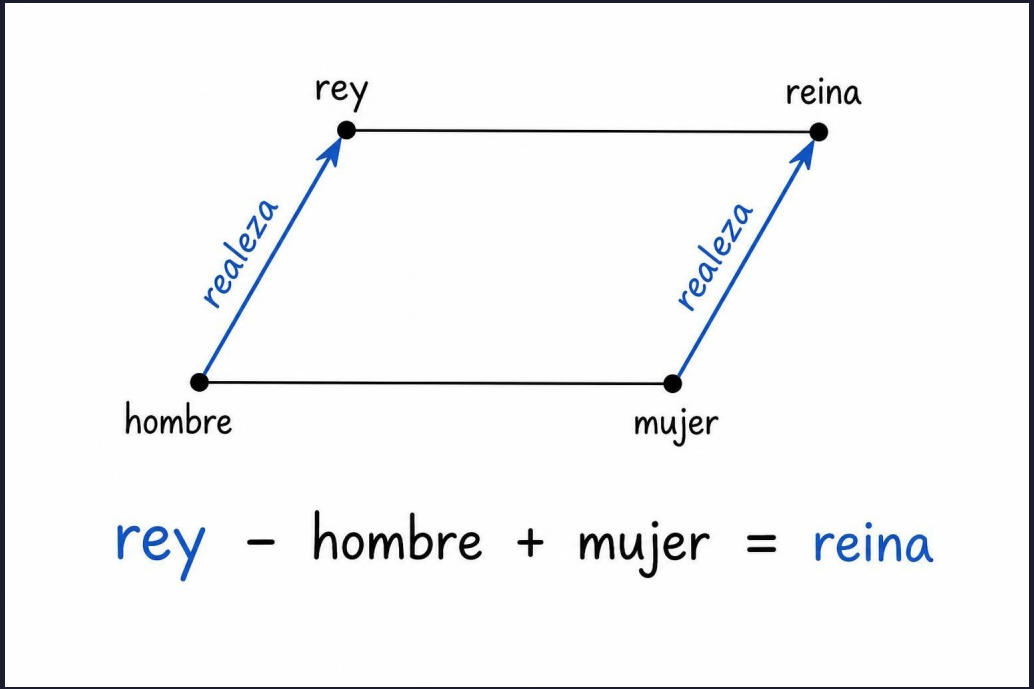
Muchas frases con la palabra → un único vector. Cada palabra, el suyo.

El espacio semántico



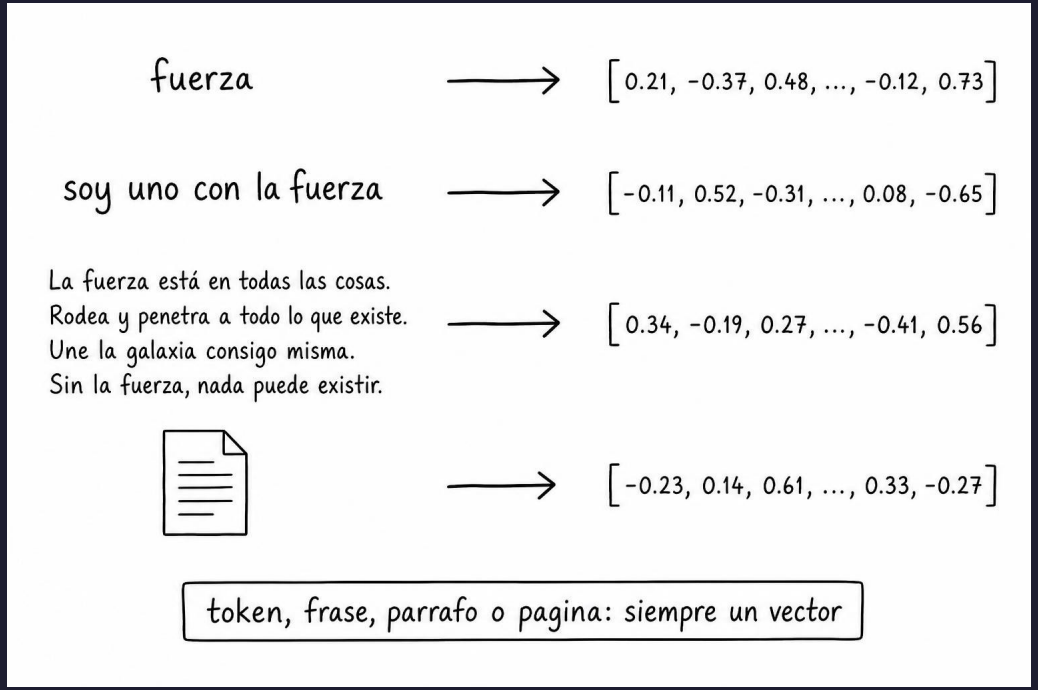
Palabras de contexto similar acaban cerca.

Aritmética semántica



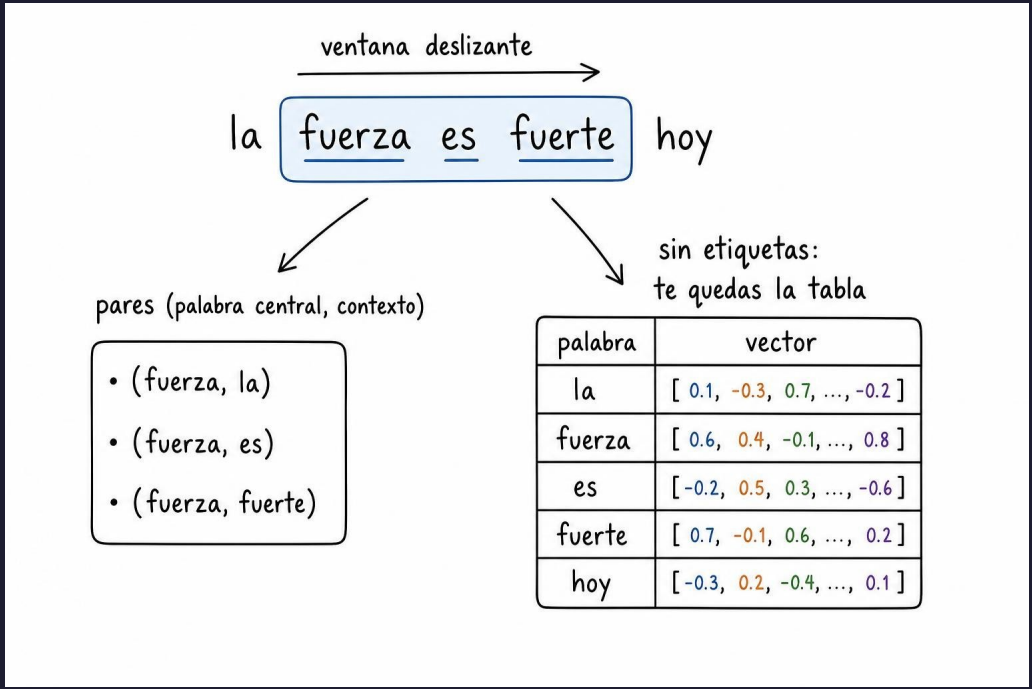
Las direcciones del espacio codifican conceptos.

Granularidad del embedding



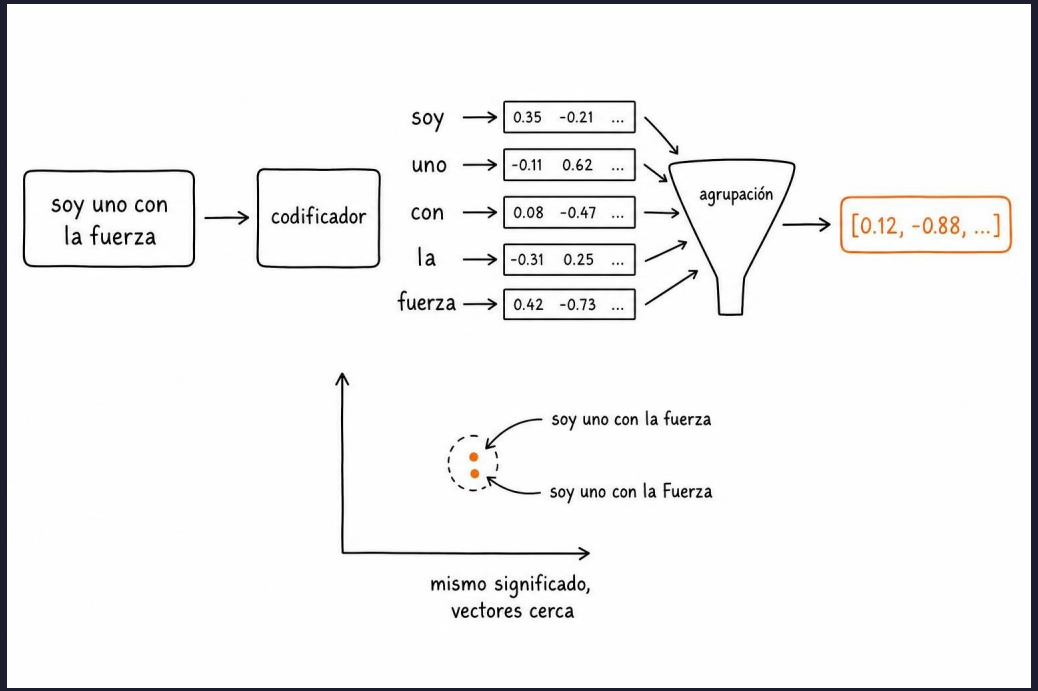
Token, frase, párrafo o página: siempre UN vector.

Cómo se entrena · token



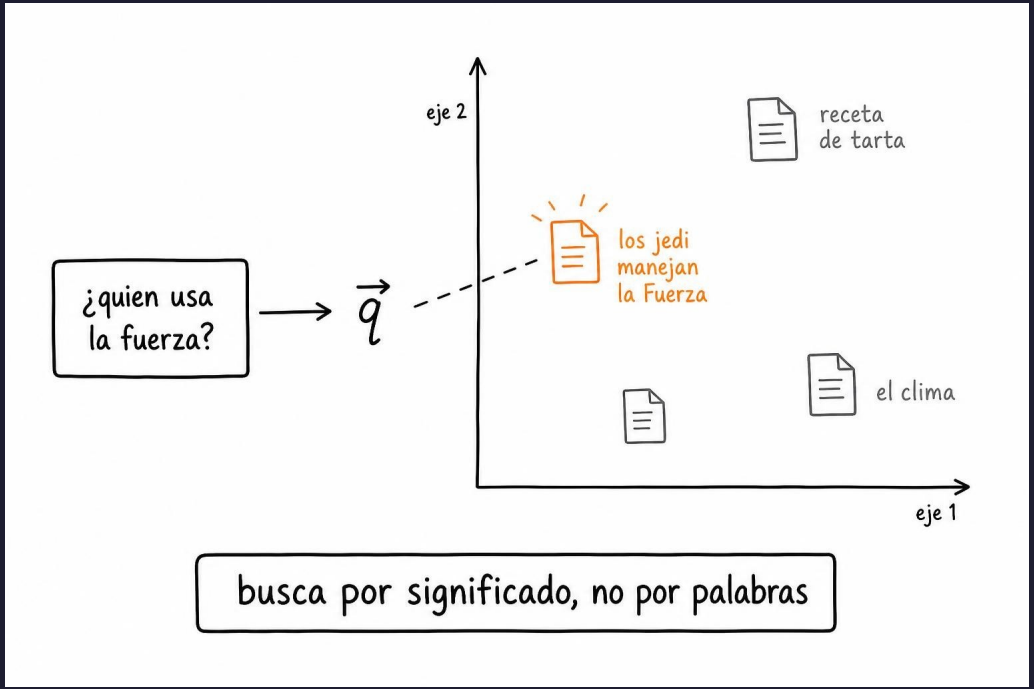
Sin etiquetas: el texto se etiqueta solo.

Cómo se entrena · frase



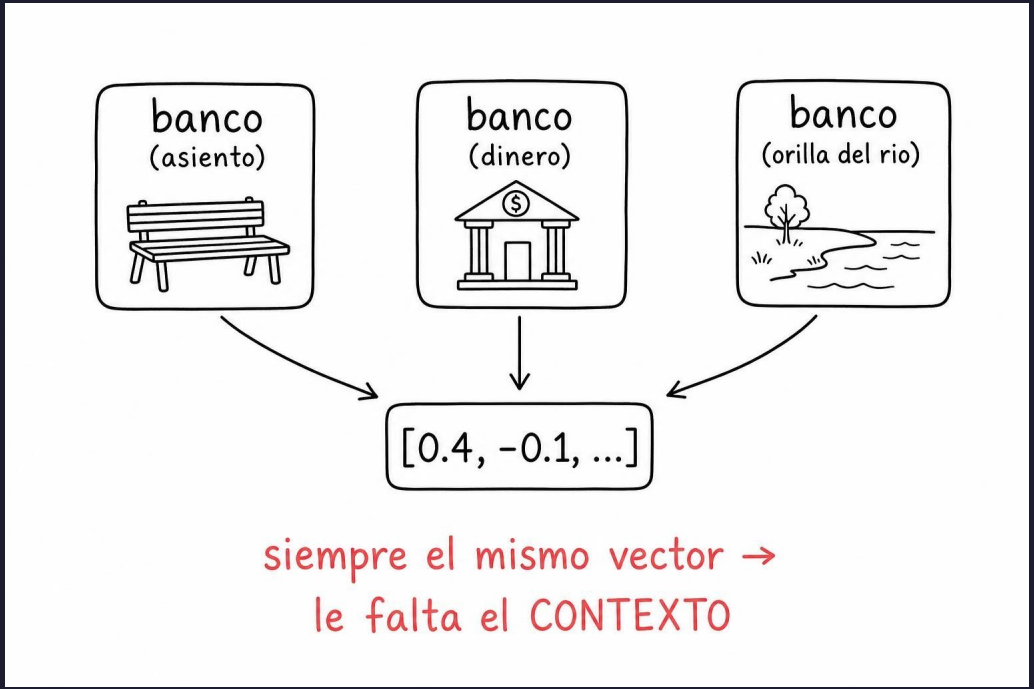
Le metes una frase, devuelve UN vector (contrastive).

Búsqueda semántica (RAG)



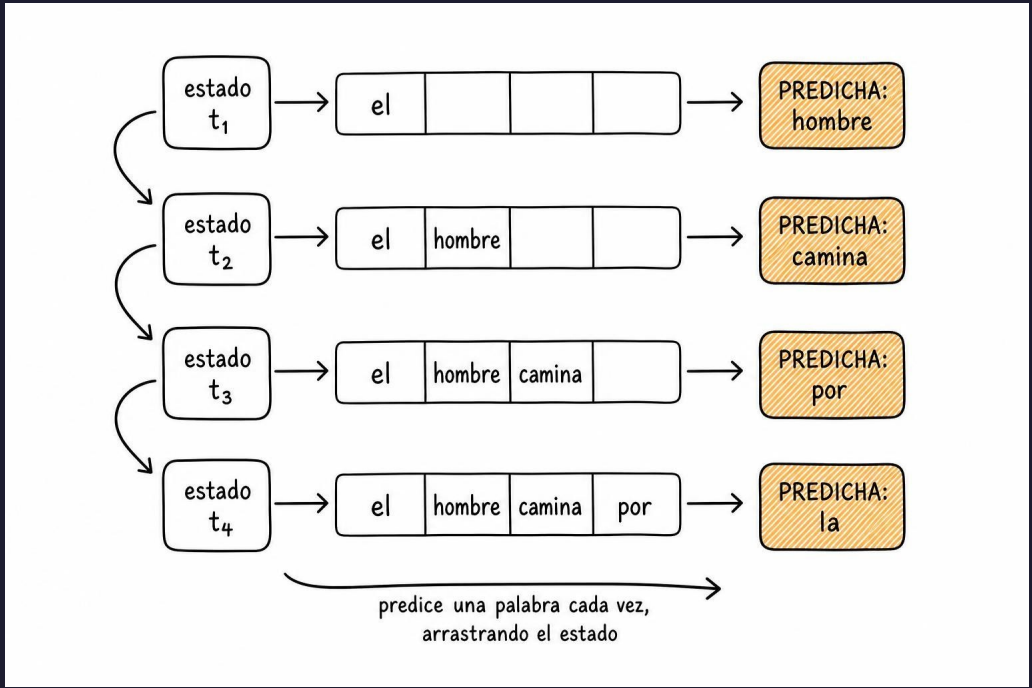
Encuentra por significado, no por palabras. Base de RAG (clase 4).

El límite de Word2Vec



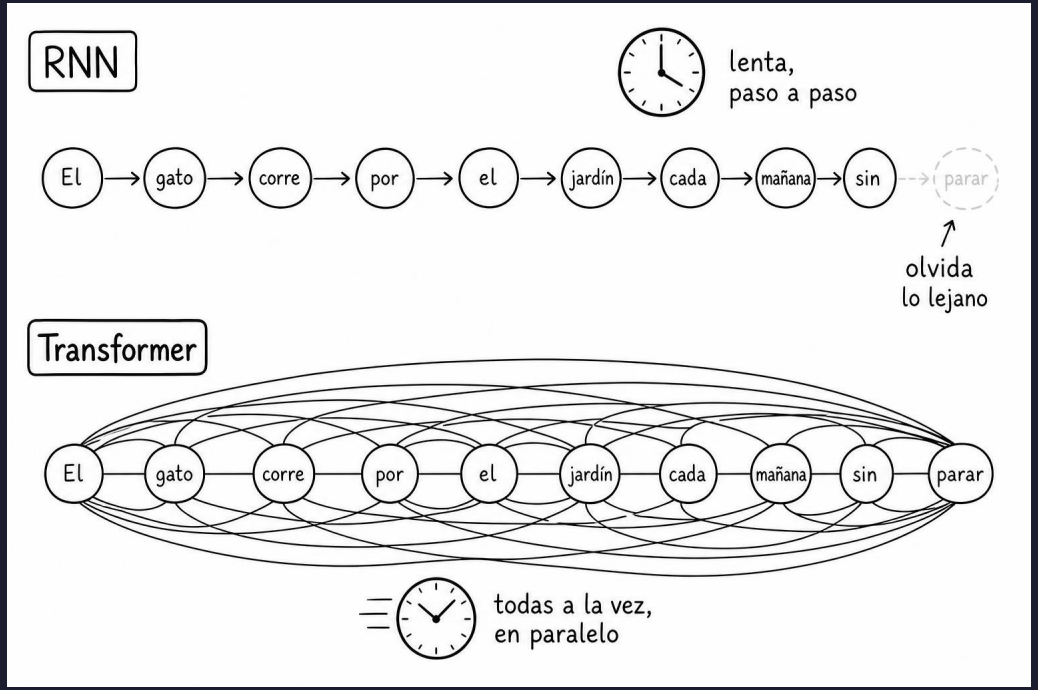
Vector fijo, sin contexto → lo resuelve el Transformer.

RNN: generar palabra a palabra



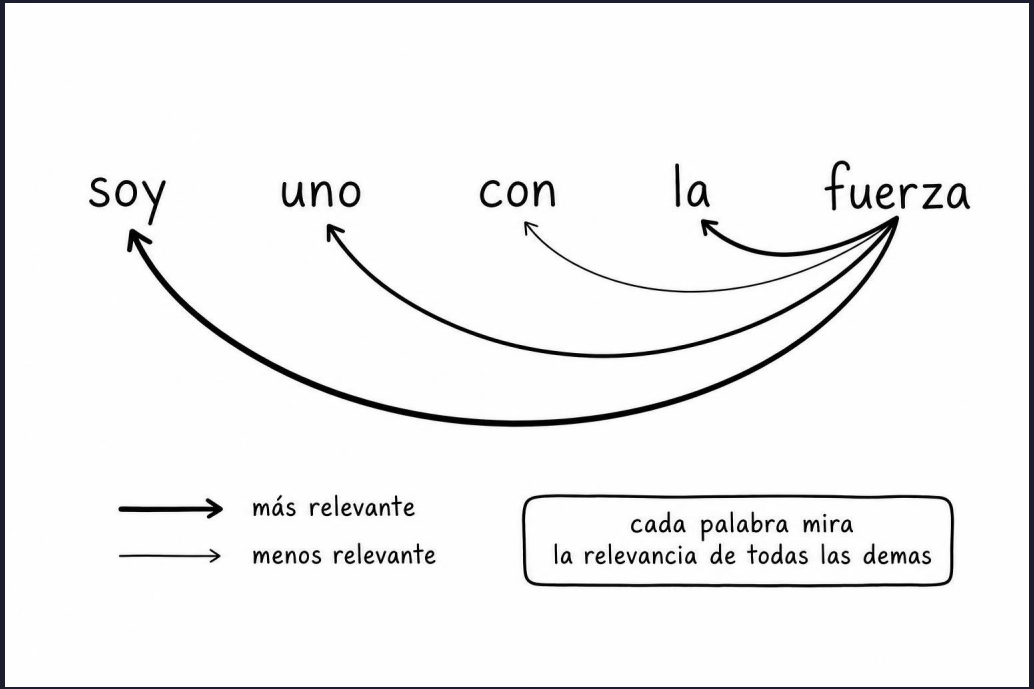
Así se generaba antes: secuencial, arrastrando el estado.

El problema de las RNN



Lenta (paso a paso) y olvida lo lejano.

Llegan los Transformers



Miran la relevancia de TODAS las palabras a la vez.

Attention is all you need (2017)



Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.com

Llion Jones* Google Research llion@google.com

Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin* ‡ illia.polosukhin@gmail.com

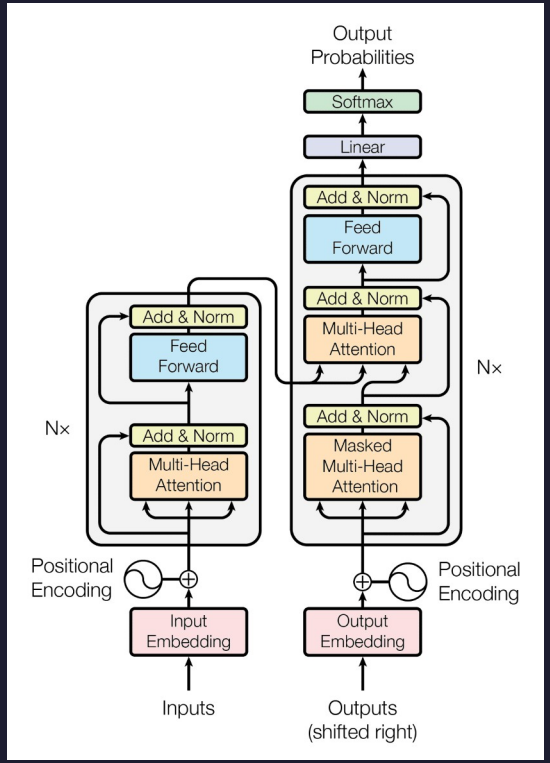
Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.0 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature.

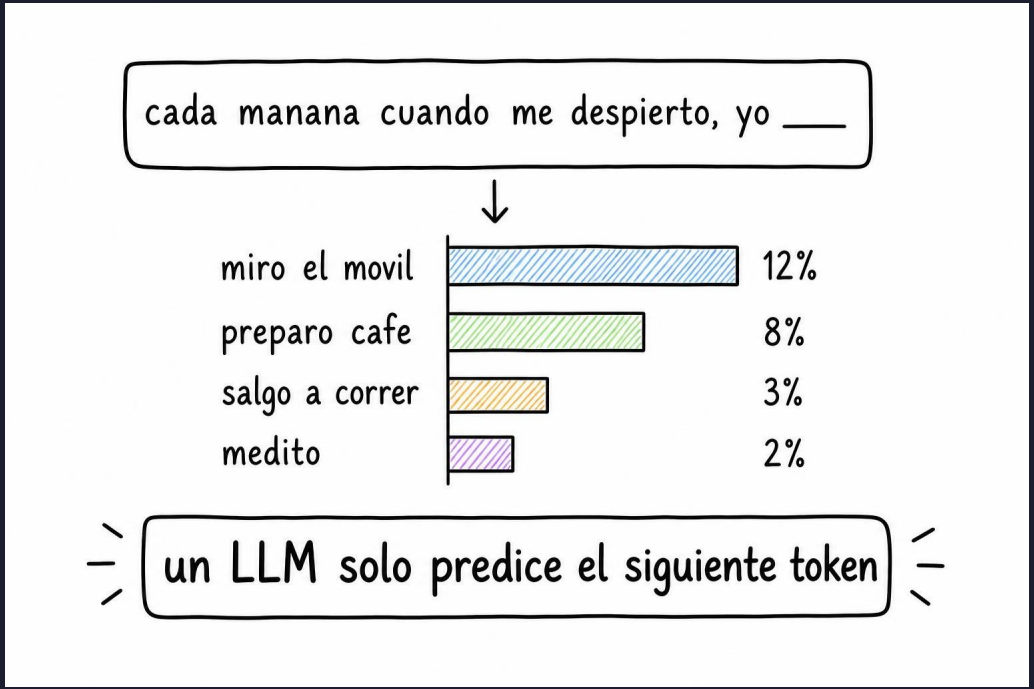
El paper de Google (2017) que lo cambió todo.

La arquitectura original (2017)

- Encoder (izquierda): lee y ENTIENDE el input.
- Decoder (derecha): GENERA la salida, token a token.
- Positional encoding: añade el orden de las palabras.
- Cada bloque se repite N× (atención + feed-forward).
- **Hoy GPT usa solo la mitad derecha: decoder-only.**

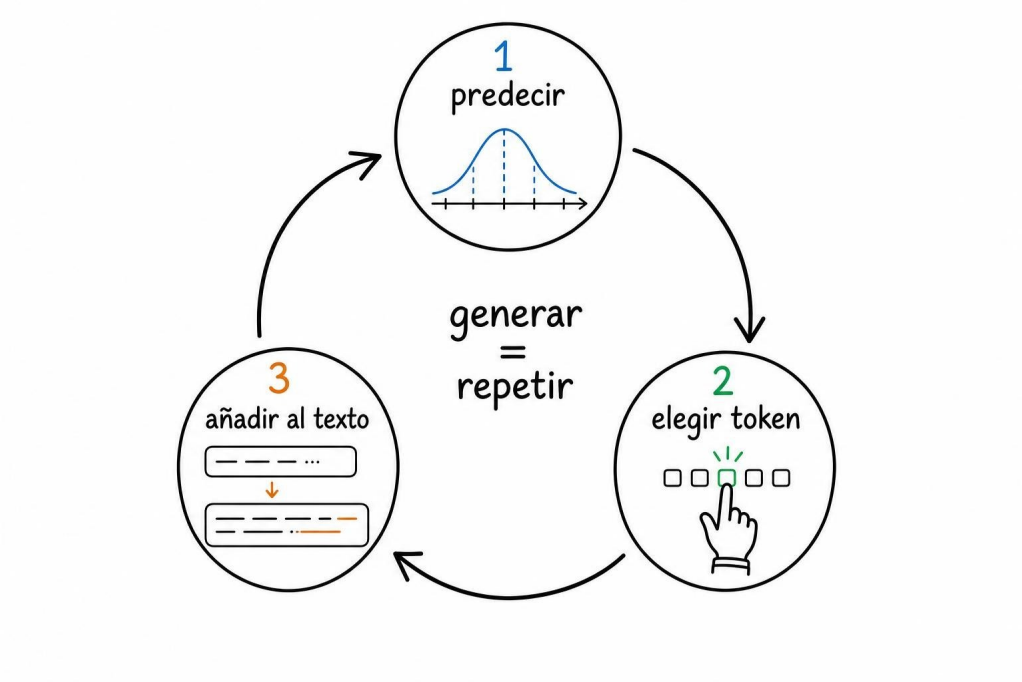


¿Qué hace de verdad un LLM?



Solo predice el siguiente token. Nada más.

El loop: predecir y repetir



Generar = predecir, elegir, añadir, repetir.

Loros estocásticos



Coherencia sin comprensión: la gran advertencia.

El Transformer de un vistazo

El objetivo: enriquecer cada palabra con su contexto para predecir la siguiente.



El bloque Atención + MLP se repite N veces. Volvemos a este mapa en cada paso.

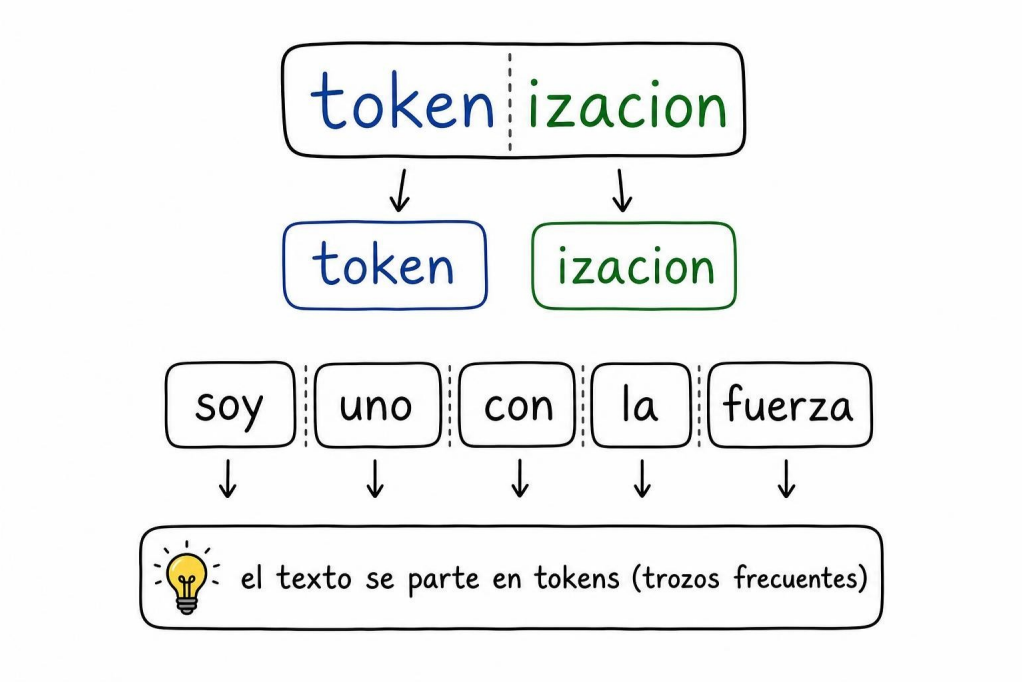
Recorriendo el mapa: Tokenización

Paso 1 de 5 · partir el texto en piezas.



El bloque Atención + MLP se repite N veces. Volvemos a este mapa en cada paso.

Tokenización



El texto se parte en tokens (trozos frecuentes).

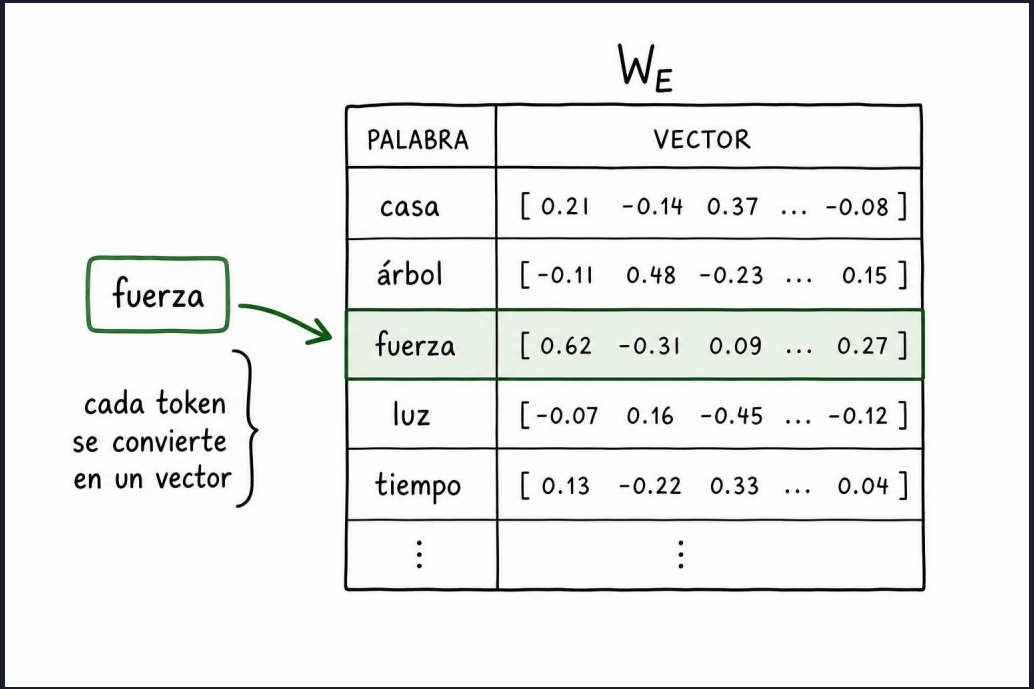
Recorriendo el mapa: Embeddings

Paso 2 de 5 · cada token a un vector.



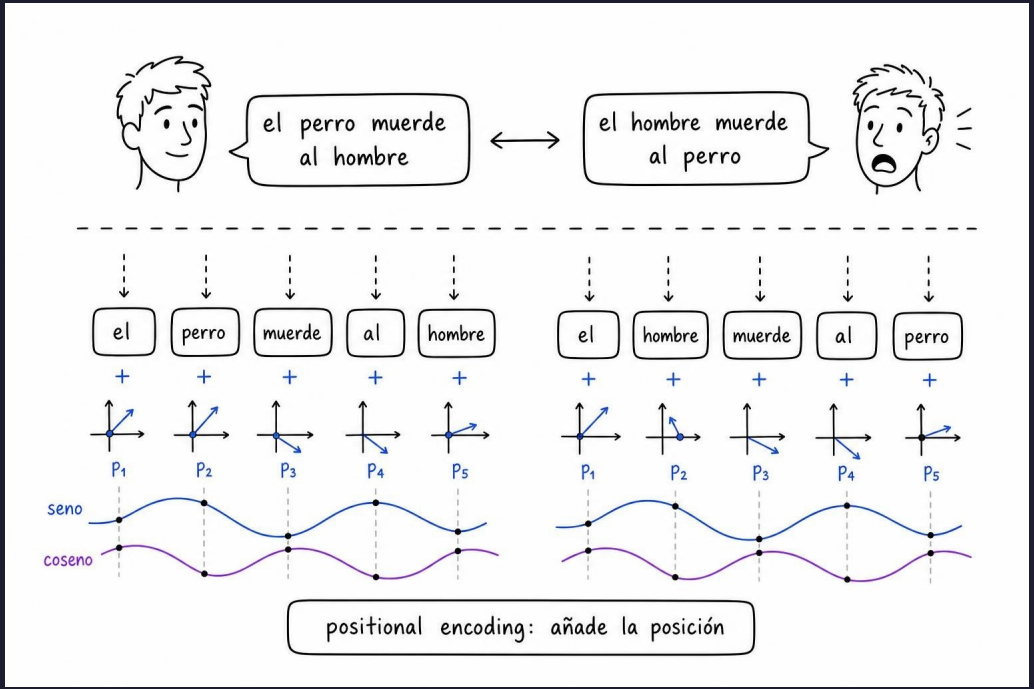
El bloque Atención + MLP se repite N veces. Volvemos a este mapa en cada paso.

Tokens → Embeddings



Cada token se convierte en un vector entrenable.

Positional Encoding



El orden importa: se suma la posición a cada token.

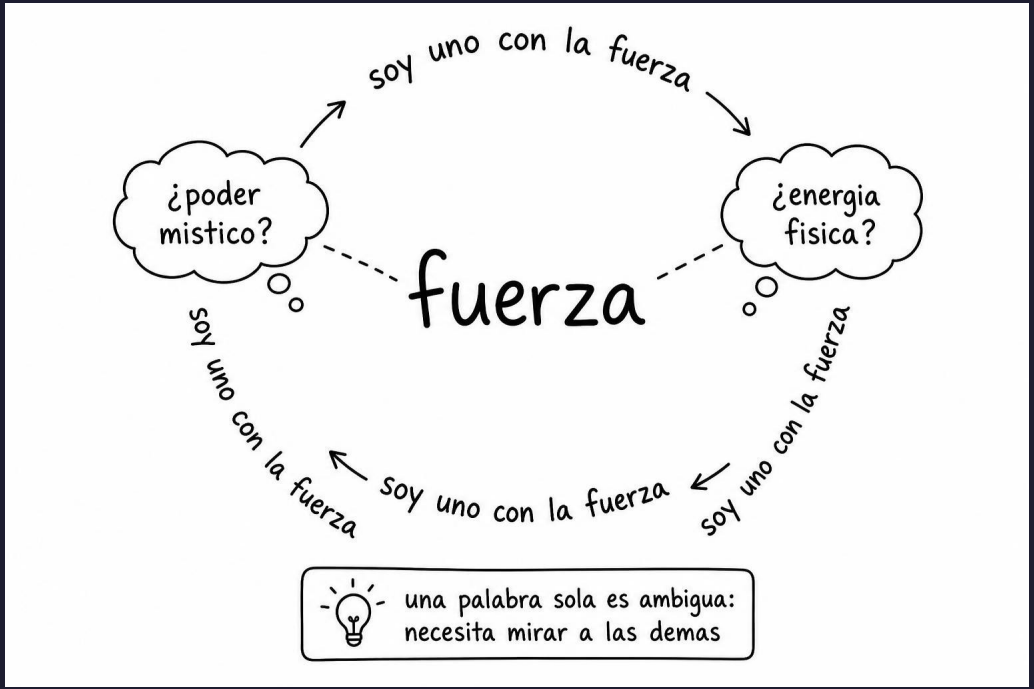
Recorriendo el mapa: Atención

Paso 3 de 5 · el corazón del Transformer.



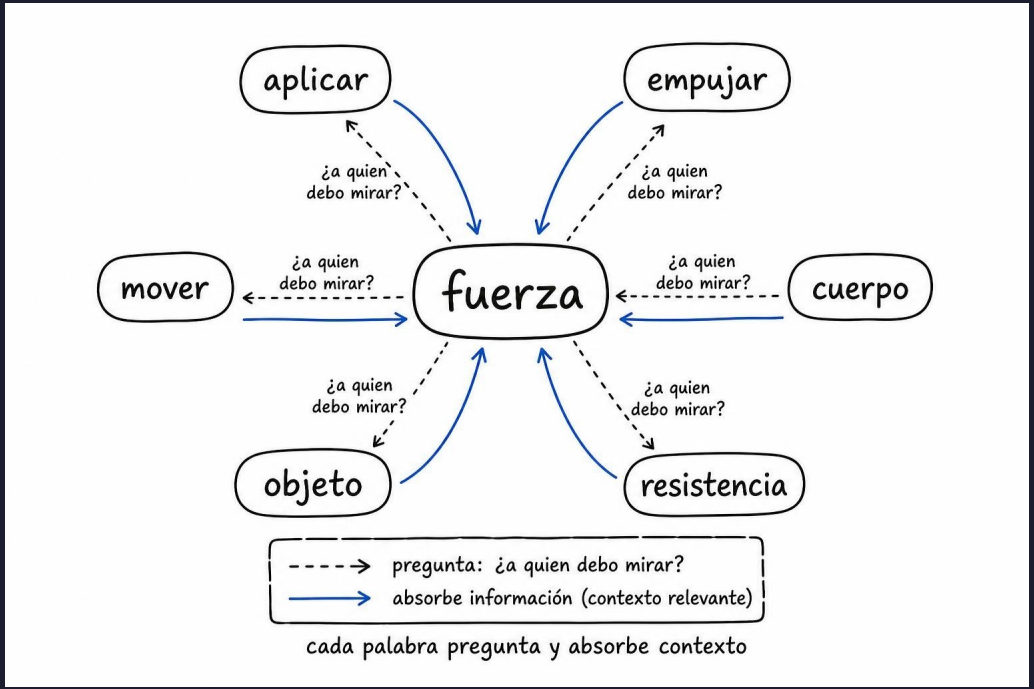
El bloque Atención + MLP se repite N veces. Volvemos a este mapa en cada paso.

Atención · el problema



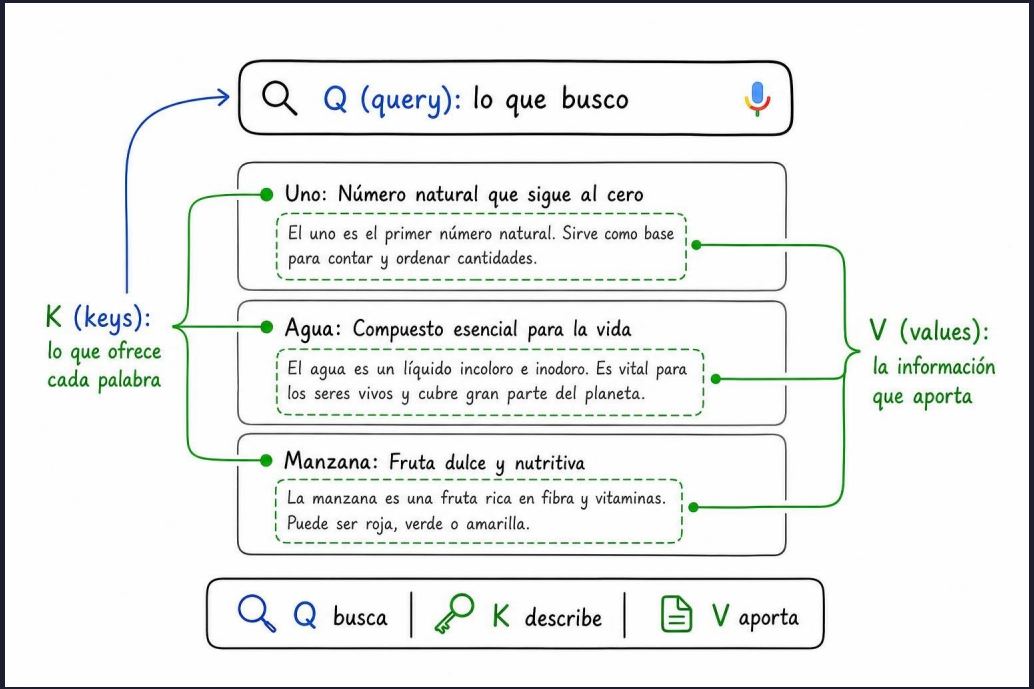
Una palabra sola es ambigua: necesita mirar a las demás.

Atención · la idea



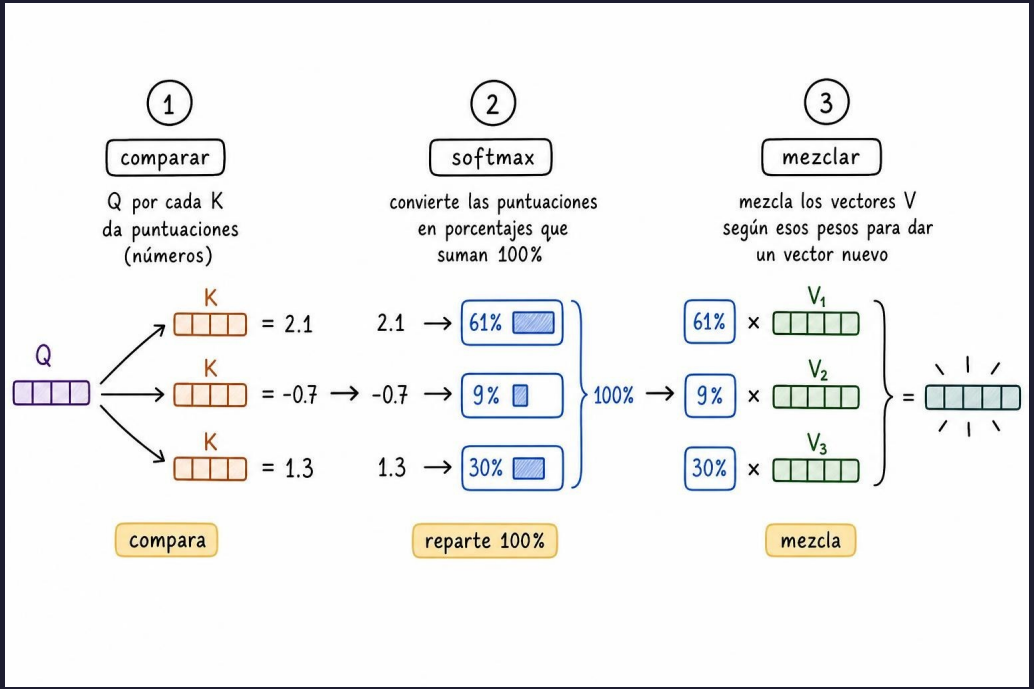
Cada palabra pregunta a quién mirar y absorbe contexto.

Q, K, V · como un buscador



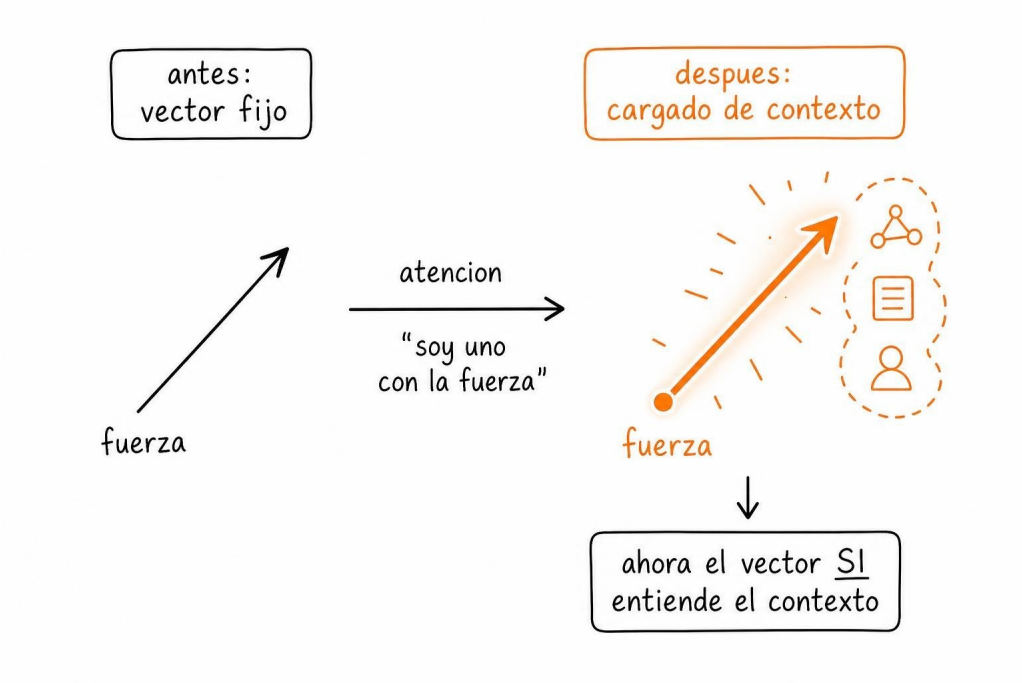
Q busca, K describe, V aporta la información.

El cálculo, en 3 pasos



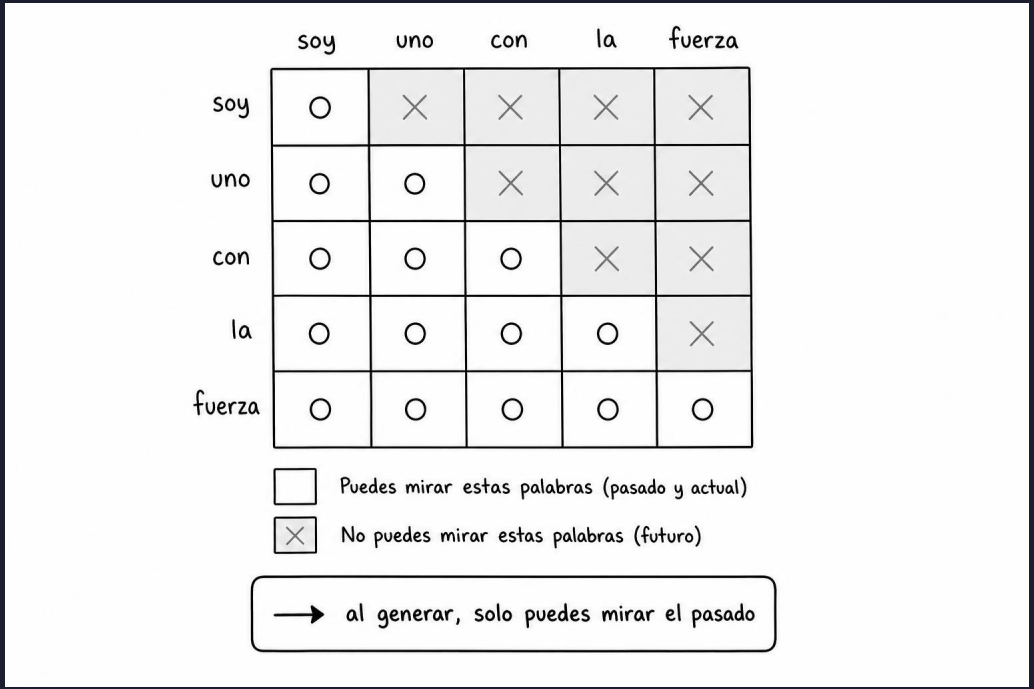
Compara (Q·K), reparte 100% (softmax), mezcla (V).

El momento clave



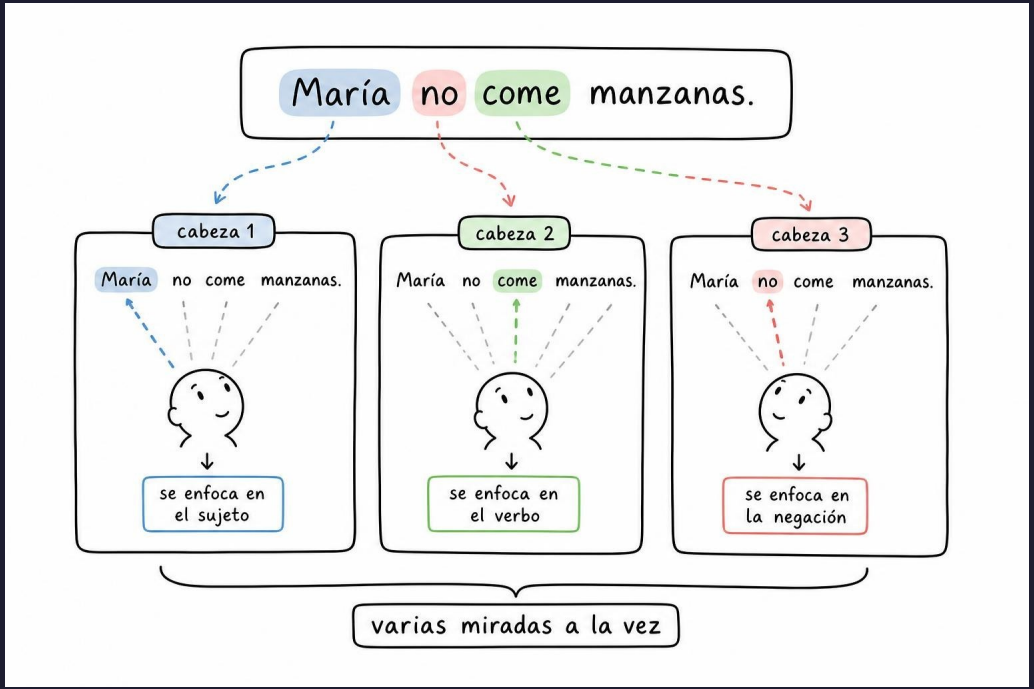
El vector de 'fuerza' ya NO es fijo: ahora carga el contexto.

Atención causal (masked)



Al generar solo puedes mirar el pasado.

Multi-head attention



Varias cabezas miran cosas distintas a la vez.

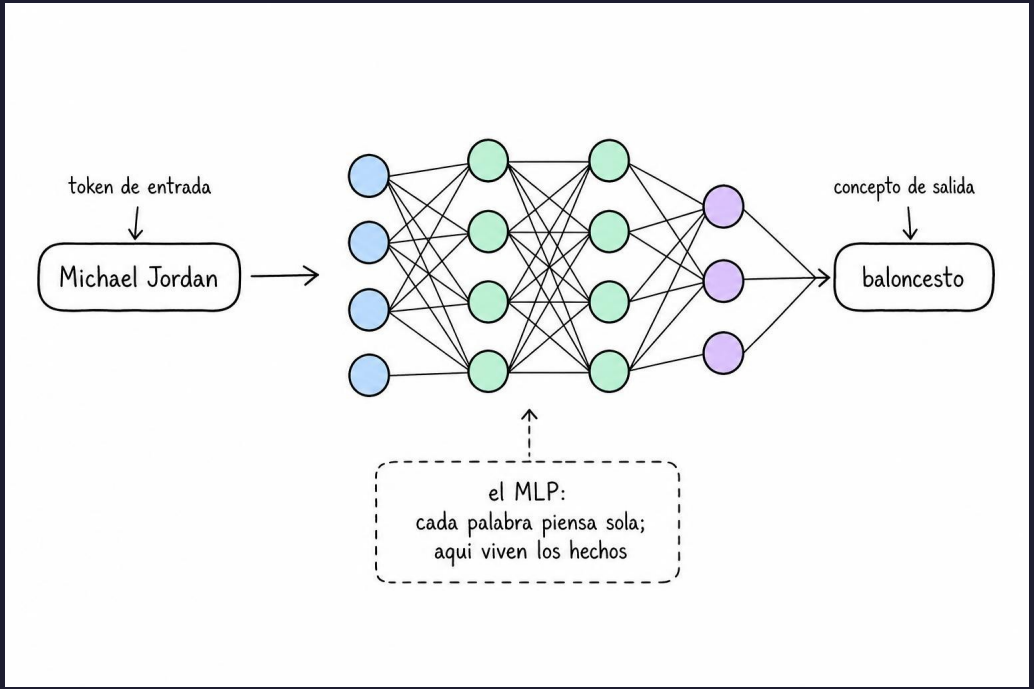
Recorriendo el mapa: MLP

Paso 4 de 5 · cada palabra piensa por su cuenta.



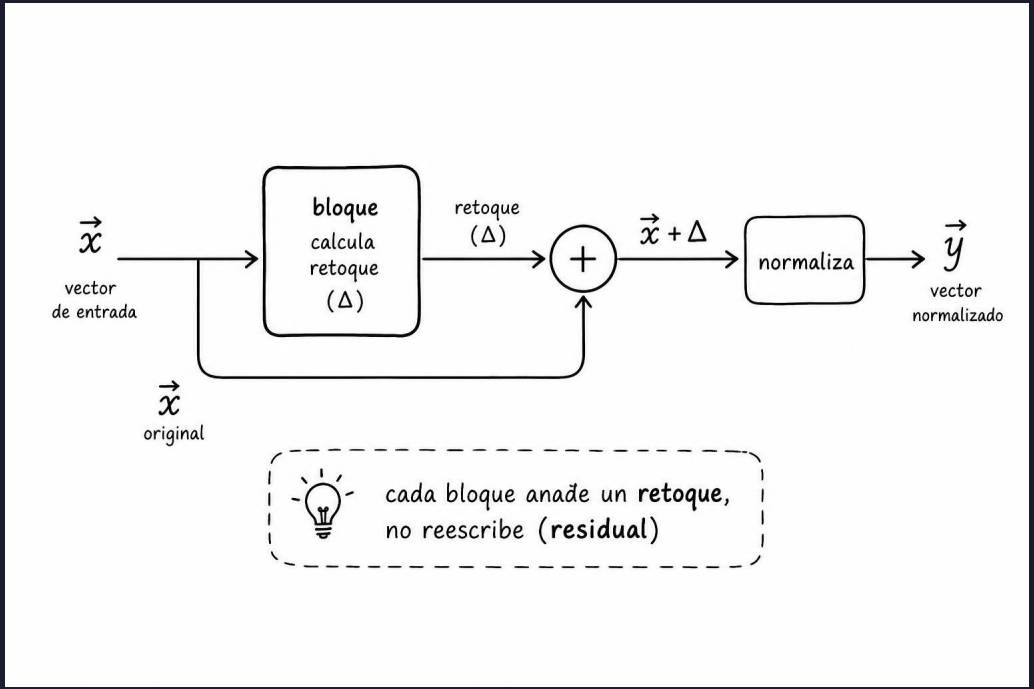
El bloque Atención + MLP se repite N veces. Volvemos a este mapa en cada paso.

El MLP: aquí viven los hechos



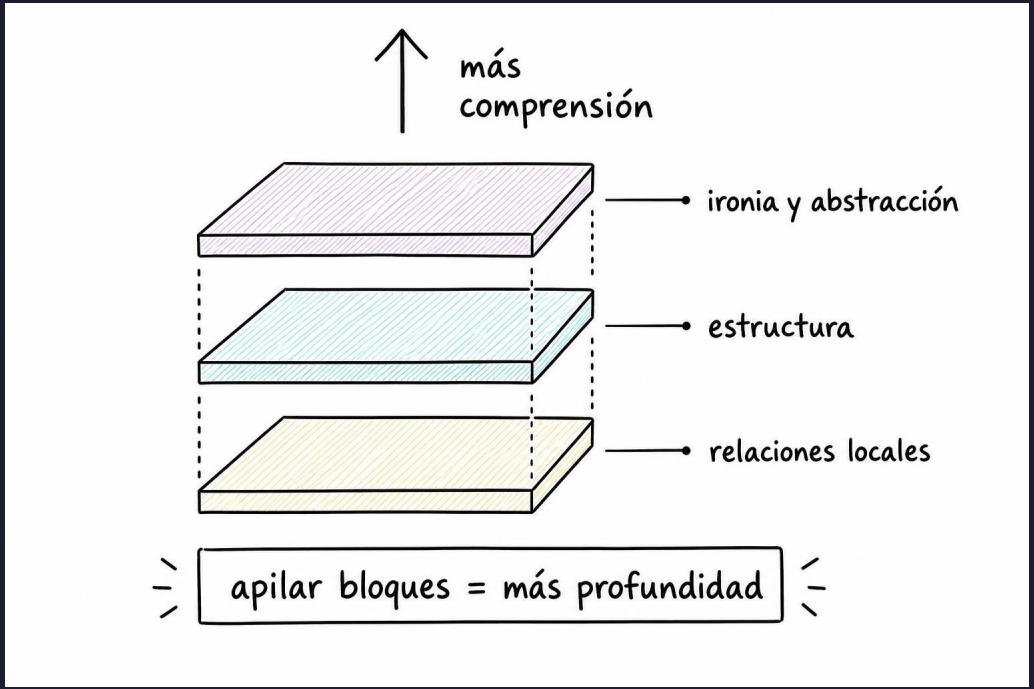
2/3 de los parámetros. Recuerda hechos del mundo.

Add & Norm (residual)



Cada bloque añade un retoque, no reescribe.

Apilar N capas



De lo local a lo abstracto: la profundidad da comprensión.

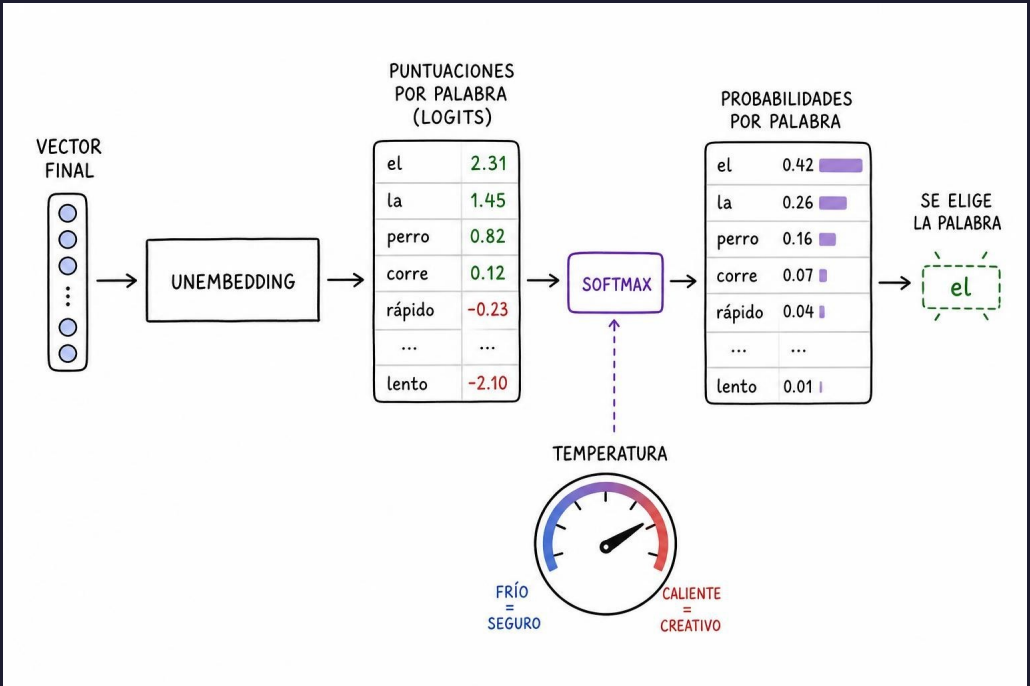
Recorriendo el mapa: Unembedding

Paso 5 de 5 · del vector a la palabra.



El bloque Atención + MLP se repite N veces. Volvemos a este mapa en cada paso.

Unembedding · softmax · temperatura



Del vector final a la siguiente palabra.

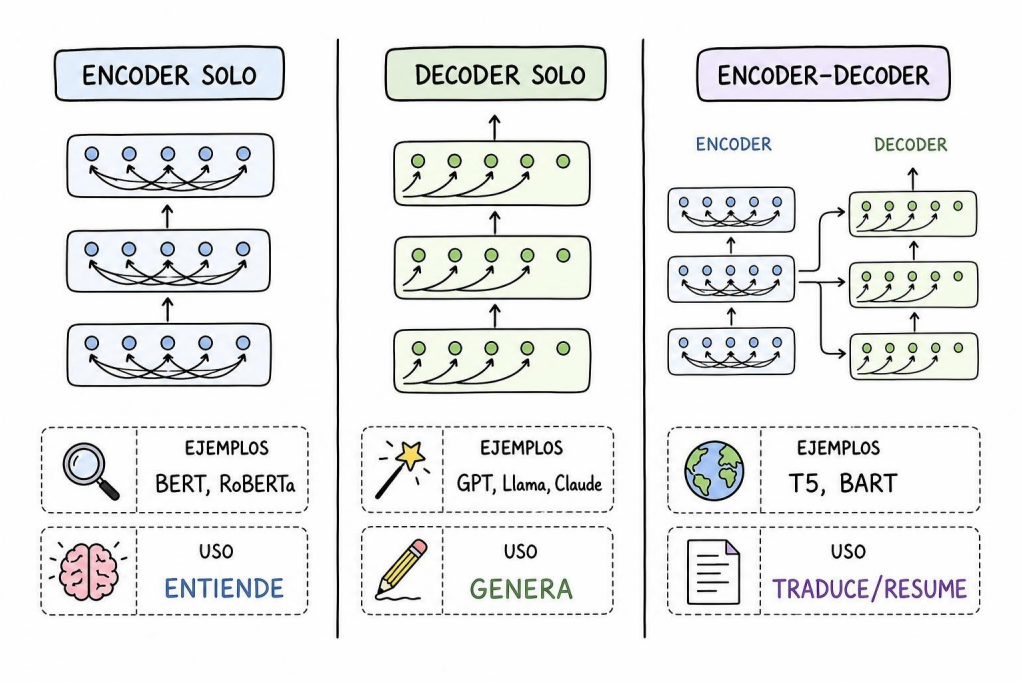
Recap del flujo completo

Tokens → Embeddings → Atención → MLP (xN) → Unembedding.



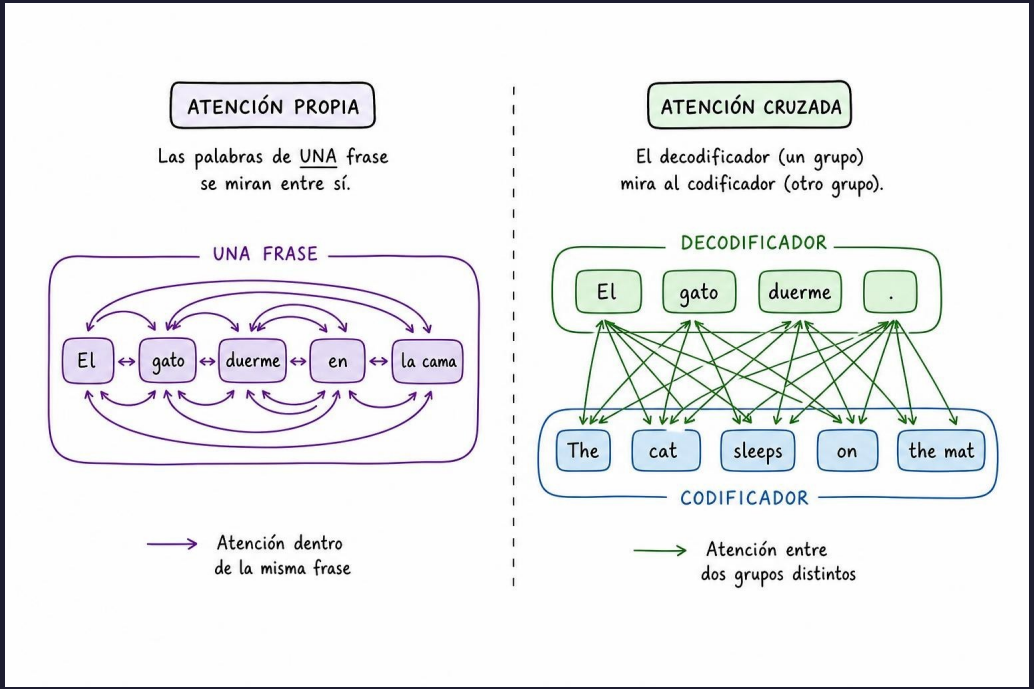
El bloque Atención + MLP se repite N veces. Volvemos a este mapa en cada paso.

Las 3 familias del Transformer



Encoder entiende · Decoder genera · Encoder-decoder traduce.

Self-attention vs cross-attention



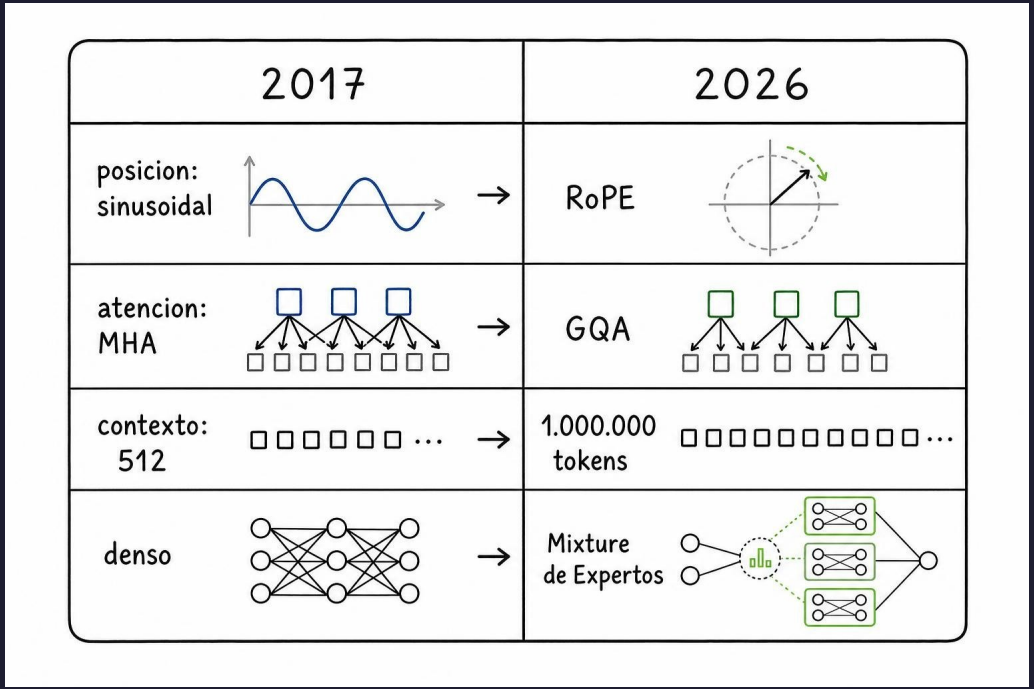
Mirarse a sí mismo vs mirar al encoder.

GPT-3 en números

175.000 M	96	96	12.288
parámetros	capas	cabezas	dimensiones

~28.000 matrices, vocab 50.257, contexto 2.048 tokens. Y eso era 2020.

Del Transformer de 2017 al de 2026



RoPE · GQA · contexto de 1M · Mixture of Experts.

2026 · Modelos de decisión

No todo Transformer genera texto. Jev (TypeSafe, sept. 2026) solo decide.

	LLM (GPT)	Modelo de decisión (Jev)
Salida	Softmax sobre todo el vocabulario	Una distribución sobre las opciones que tú defines
Cómo decide	En bucle: predecir, elegir, añadir, repetir (con temperatura y sampling)	Una sola pasada: sin bucle ni sampling
Latencia	Crece con cada token que genera	70-500 ms (~100 ms)
Coste de output	Pagas cada token generado	Gratis
Error de tipo	Posible: formato roto o etiqueta inexistente	Imposible: siempre una de tus opciones
¿Escribe texto?	**Sí**	**No**

Analogía de comportamiento: TypeSafe no ha publicado la arquitectura de Jev.

keep coding

Jev · tres tipos de pregunta

Ej.: Choice sobre «soy uno con la fuerza»: ¿positivo, neutro o negativo?

Tipo	Qué responde	Qué devuelve	Ejemplo de soporte
Choice	Elige entre tus opciones (hasta 255)	choice + probabilities + confidence	Categoría del ticket
Score	Escala de 2 a 10 niveles	float + probabilities	Urgencia 1-5
Noul	Sí o no (Bernoulli)	Probabilidad de 0 a 1	¿Tiene pasos de reproducción?

Úsalo para: clasificar, enrutar, puntuar, priorizar
No para: escribir, calcular, decisiones de alto impacto

0,042 $/1M tokens de input, output gratis.

Fuente: TypeSafe AI · docs.typesafe.ai · acceso anticipado

Mini-GPT desde cero (lab)

- Construir un Transformer decoder-only en PyTorch, desde cero.
- Entrenar sobre un texto (El Quijote) y generar texto nuevo.
- Ver en código tokens, embeddings, atención y el loop de generación.
- → vamos al notebook.

Recursos para seguir

- poloclub.github.io/transformer-explainer · visualiza un GPT real
- 3Blue1Brown · serie de vídeos sobre Transformers y atención
- jalammar.github.io/illustrated-gpt2 · The Illustrated GPT-2
- Vaswani et al. (2017) · 'Attention is all you need'